

Algorithms & Computability

How to Actually Solve the Exercises

A Skills Guide for Tutorials 1–4

AC Exercise Method Guide

Spring 2026

Contents

1	The Real Problem (Read This First)	2
1.1	The 9 Exercise Types (Map)	2
1.2	What You Must Know Cold (Before Anything Else)	2
2	Type 1: Definition Checking	4
2.1	The Method (Step by Step)	4
2.2	Worked Example: “Can a DTM write the blank symbol?”	4
2.3	Worked Example: “Is this definition of CE correct?”	4
2.4	The Checklist for Definition-Checking Questions	5
2.5	Counterexample Strategy (for True/False Claims)	5
3	Type 2: Mechanical Tracing (Runs)	7
3.1	The Method (Step by Step)	7
3.2	Worked Example: A Full Trace	7
3.3	Common Mistakes in Tracing	9
4	Type 3: DTM/NTM Construction	10
4.1	The Method (Step by Step)	10
4.2	The Key Insight: States = Memory	11
4.3	Worked Example: First Letter = Last Letter	11
4.4	State Diagram Visualization	12
4.5	The Answer Format the Grader Expects	13
4.6	NTM Construction: When and How	13
5	Type 4: Simulation / Robustness Proofs	14
5.1	The Method	14
5.2	Worked Example: The “Stay” Direction	14
6	Type 5: Spot the Error in Proofs	16
6.1	The Method	16
7	Type 6: Closure Properties	17
7.1	The Method	17
8	Type 7: Computability Classification	18
8.1	The Decision Flowchart	18
8.2	Writing the DTM for Computable Cases	19

9	Type 8: Reduction Proofs	20
9.1	What a Reduction Actually Is	20
9.2	The Method	20
9.3	Worked Example: NEP is Not Computable	21
10	Type 9: CE vs Computable Arguments	22
10.1	The Three Key Tools	22
10.2	Worked Example: HP is CE	22
10.3	Worked Example: $\overline{AP}$ is Not CE	23
11	Putting It All Together: Your Study Plan	24
11.1	Quick Reference: Which Template for Which Exercise	24
11.2	The Answer Format Cheat Sheet	24

1 The Real Problem (Read This First)

Why You Can Understand the Lectures But Can't Solve the Exercises

The Core Problem: Understanding “what a Turing machine is” and being able to *write formal answers about Turing machines* are two completely different skills. The lectures teach you concepts. The exercises test whether you can **operate with formal definitions** — cite them precisely, construct machines from them, and build arguments using them as building blocks.

Analogy: Imagine you understand how chess pieces move. You can explain “the bishop moves diagonally.” But when someone puts a position in front of you and says “find checkmate in 3 moves” — you freeze. You *understand* chess, but you haven't practiced *playing* chess. The exercises are the game. This guide teaches you the moves.

What this guide does:

- Identifies the **9 exercise types** that appear in Tutorials 1–4
- For each type, gives you a **step-by-step template** you can follow
- Shows you **what a correct answer looks like** and what the grader expects
- Does NOT give you solutions — it teaches you the **method** to find solutions yourself

The exercises are not testing whether you're smart. They're testing whether you can follow a formal recipe. This guide gives you the recipes.

1.1 The 9 Exercise Types (Map)

Every exercise in Tutorials 1–4 falls into one of these categories:

Type	Name	Where It Appears
1	Definition Checking	T1-Ex1, T1-Ex2, T2-Ex3, T3-Ex1
2	Mechanical Tracing (Runs)	T1-Ex3, T1-Ex4 runs, T1-Ex5 runs
3	DTM/NTM Construction	T1-Ex4, T1-Ex5, T2-Ex4, T3-Ex3
4	Simulation / Robustness	T2-Ex1, T2-Ex2
5	Spot the Error	T3-Ex2
6	Closure Properties	T2-Ex5, T3-Ex5
7	Computability Classification	T4-Ex1, T4-Ex2
8	Reduction Proofs	T4-Ex5
9	CE vs Computable Arguments	T3-Ex4, T4-Ex3, T4-Ex4

1.2 What You Must Know Cold (Before Anything Else)

The 5 Definitions You Must Be Able to Write From Memory

Before you can solve *any* exercise, you need to be able to write these 5 things from memory on a blank sheet of paper. Not “roughly know” — **write them out exactly**. If you can't, nothing else in this guide will help.

1. **DTM definition:** $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with $\delta : (Q \setminus \{t, r\}) \times \Gamma \rightarrow Q \times \Gamma \times \{-1, +1\}$

Breaking it down:

- Q = finite set of states
 - Σ = input alphabet (the symbols that can appear in the input)
 - Γ = tape alphabet (always $\Sigma \subset \Gamma$ and $\sqcup \in \Gamma$ but $\sqcup \notin \Sigma$)
 - $s \in Q$ = start state
 - $t \in Q$ = accept state
 - $r \in Q$ = reject state, $t \neq r$
 - δ = transition function: takes (current state, current symbol) and outputs (new state, symbol to write, direction to move)
 - δ is only defined for states that are NOT t or r (the machine stops in those)
2. **Configuration:** $[q, \tau, \ell]$ where q = state, τ = tape content (function $\mathbb{N} \rightarrow \Gamma$), ℓ = head position. Simplified notation: $[q, \text{tape contents} \dots, \ell]$.
 3. **Computable language:** L is computable $\iff$ there exists a halting DTM M such that $L(M) = L$. (Halting = M stops on *every* input, both in L and not in L .)
 4. **Computably-enumerable (CE) language:** L is CE $\iff$ there exists a DTM M (not necessarily halting) such that $L(M) = L$. (If $w \in L$, M accepts. If $w \notin L$, M may reject OR loop forever.)
 5. **NTM:** Same as DTM but $\delta : (Q \setminus \{t, r\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{-1, +1\})$ (returns a *set* of possible transitions). Accepts if *at least one* computation path reaches t .

Quiz yourself right now: cover the definitions above and try to write them on paper. If you can't, re-read them 3 times, then try again. This is prerequisite #0.

2 Type 1: Definition Checking

What This Type Looks Like

Pattern: You're given a statement ("Can a DTM do X?" or "Is this definition of CE correct?") and you must say yes/no with justification.

Where it appears: T1-Ex1, T1-Ex2, T2-Ex3, T3-Ex1

Analogy: Think of it like a lawyer reading a contract. The question is: "Does the contract allow this?" You don't give your opinion — you point to the specific clause (definition) that says yes or no. The answer is ALWAYS in the definition.

2.1 The Method (Step by Step)

Template: Definition Checking

1. **Write down the relevant formal definition** on your scratch paper. Not from memory of "what it means" — write the actual mathematical definition with all its symbols.
2. **Identify which part of the definition the question targets.** Is it about δ ? About Γ vs Σ ? About halting vs non-halting?
3. **Answer by pointing to the definition.** Your answer should literally say: "Yes/No, because the definition states that [quote the relevant part]."
4. **If the question asks about a claim (true/false):** Try to either prove it (using the definition) or find a counterexample. Counterexamples are usually simpler.

2.2 Worked Example: "Can a DTM write the blank symbol?"

Walking Through T1-Ex1.1

Question: Can a DTM ever write the blank symbol $\sqcup$ on its tape?

Step 1: Write down the relevant definition.

The transition function has type: $\delta : (Q \setminus \{t, r\}) \times \Gamma \rightarrow Q \times \Gamma \times \{-1, +1\}$

The output includes $Q \times \Gamma \times \{-1, +1\}$. The second component is what gets *written* to the tape.

Step 2: Which part matters?

The question asks about writing $\sqcup$. The written symbol comes from Γ (the tape alphabet).

Step 3: Answer by citing the definition.

Yes, a DTM can write $\sqcup$, because $\sqcup \in \Gamma$ (by definition, the blank symbol is always in the tape alphabet), and δ outputs an element of Γ as the symbol to write.

Notice: the answer is just "look at the type signature of δ , see that $\sqcup \in \Gamma$, done." No creativity needed.

2.3 Worked Example: "Is this definition of CE correct?"

Walking Through T1-Ex2 (Checking Formal Statements)

Question: "A language L is computably-enumerable if the language halts in the state t whenever $w \in L$." Correct or wrong?

Step 1: Write down the correct definition.

L is CE if there exists a DTM M such that M , given input w , halts in t if $w \in L$ (and may not halt if $w \notin L$).

Step 2: Compare word by word.

- The given statement says “**the language** halts” — but *languages don’t halt*. Only *Turing machines* halt. A language is just a set of strings.
- The given statement also doesn’t say “there exists a DTM M ” — it’s missing the quantifier.

Step 3: Write the answer.

WRONG: “the language halts” is nonsensical — a language is a set of strings, it cannot halt. Turing machines halt, not languages. Also, the definition requires “there exists a DTM M such that...”

The trick for these: read EVERY word of the given statement and check it against the definition. The errors are usually: wrong subject (“language” instead of “DTM”), wrong quantifier (“every” instead of “exists”), or missing pieces.

2.4 The Checklist for Definition-Checking Questions

Common Traps to Watch For

When checking if a statement about CE/computable/NTM is correct, these are the things that are usually wrong:

Trap	What’s Wrong
“The language halts/accepts/rejects”	Languages don’t do things, DTMs do
“Every DTM M ...”	Should be “there exists a DTM M ”
“If and only if $w \in L$ ” for CE	CE only requires “if”, not “only if”
“More than one computation” for NTM	NTM needs “at least one”, not “more than one”
“Every computation accepts” for NTM	NTM needs “at least one”, not “every”
“ M rejects or does not halt, then do X”	You <i>cannot detect</i> non-halting
“ M is a member of L ”	L contains strings, not machines

90% of these questions test whether you confuse “for all” with “there exists” or whether you apply predicates to the wrong objects.

2.5 Counterexample Strategy (for True/False Claims)

Template: Disproving a Claim

When a question says “Is this claim true? If not, give a counterexample” (like T3-Ex1):

1. **First instinct check:** Does the claim feel too strong? “Every subset of a computable language is computable” — that would mean computability is “inherited” by all subsets, which would be very powerful.
2. **Try the extreme case:** What’s the biggest/simplest language you know?
 - Σ^* is computable (accept everything)
 - $\emptyset$ is computable (reject everything)
 - Every language L is a subset of Σ^*

3. **Use a known non-computable language:** If the claim says “every subset of L is computable”, try $L = \Sigma^*$ and pick any non-computable $L' \subseteq \Sigma^*$ (like HP). Since $HP \subseteq \Sigma^*$ and HP is not computable, the claim is false.

4. **Write the counterexample clearly:**

- “Let $L = \Sigma^*$. This is computable (accepted by a DTM that always accepts).”
- “Let $L' = HP$. Then $L' \subseteq L$ (every language is a subset of Σ^*).”
- “But HP is not computable (proven in lecture). Contradiction.”

When disproving: always try Σ^* or $\emptyset$ as your base language first, then pick HP or AP as the counterexample subset. This works surprisingly often.

3 Type 2: Mechanical Tracing (Runs)

What This Type Looks Like

Pattern: Given a DTM (as a δ table) and an input word, write out the complete run step by step.

Where it appears: T1-Ex3, T1-Ex4 (parts 3–5), T1-Ex5 (parts 2–4)

Analogy: This is like being a human debugger. You have the source code (δ table), the input, and you just step through it line by line. No thinking — just table lookup. If you can read a table, you can do this.

3.1 The Method (Step by Step)

Template: Tracing a DTM Run

Setup: You have $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ given as a table, and input $w = a_1a_2 \dots a_n$.

1. **Write the initial configuration:** $[s, a_1a_2 \dots a_n \sqcup \dots, 1]$
 - State = s (always start in s)
 - Tape = the input followed by blanks
 - Head position = 1 (leftmost cell)
2. **For each step, do exactly this:**
 - (a) Look at current state q and current symbol (the symbol at position ℓ on the tape)
 - (b) Find the row q and column for that symbol in the δ table
 - (c) Read off (q', b, d) from the table
 - (d) Write the new configuration: state becomes q' , the symbol at position ℓ on the tape changes to b , head moves to position $\ell + d$
 - (e) **Special case:** if head is at position 1 and $d = -1$, the head stays at position 1 (can't go left of the tape)
3. **Stop when** the state is t (accepted) or r (rejected).
4. **Write the full run using $\vdash_M$ notation:**
 $[\text{config}_0] \vdash_M [\text{config}_1] \vdash_M [\text{config}_2] \vdash_M \dots \vdash_M [\text{final config}]$

3.2 Worked Example: A Full Trace

Tracing the DTM from T1-Ex3 on Input aab

Given δ :

δ	a	b	$\sqcup$
s	$(h, \sqcup, +1)$	$(r, b, +1)$	$(t, \sqcup, +1)$
h	$(h, a, +1)$	$(h, b, +1)$	$(e, \sqcup, -1)$
e	$(r, a, +1)$	$(g, \sqcup, -1)$	$(r, \sqcup, +1)$
g	$(g, a, -1)$	$(g, b, -1)$	$(s, \sqcup, +1)$

Input: $w = aab$

Step-by-step:

1. **Config 0:** $[s, aab \sqcup \dots, 1]$
 State = s , symbol at position 1 = a .

Look up $\delta(s, a) = (h, \sqcup, +1)$.

$\Rightarrow$ State becomes h , write $\sqcup$ at position 1, head moves to position 2.

2. **Config 1:** $[h, \sqcup ab \sqcup \dots, 2]$

State = h , symbol at position 2 = a .

Look up $\delta(h, a) = (h, a, +1)$.

$\Rightarrow$ State stays h , write a at position 2 (unchanged), head moves to 3.

3. **Config 2:** $[h, \sqcup ab \sqcup \dots, 3]$

State = h , symbol at position 3 = b .

Look up $\delta(h, b) = (h, b, +1)$.

$\Rightarrow$ State stays h , write b at position 3 (unchanged), head moves to 4.

4. **Config 3:** $[h, \sqcup ab \sqcup \dots, 4]$

State = h , symbol at position 4 = $\sqcup$.

Look up $\delta(h, \sqcup) = (e, \sqcup, -1)$.

$\Rightarrow$ State becomes e , write $\sqcup$ (unchanged), head moves back to 3.

5. **Config 4:** $[e, \sqcup ab \sqcup \dots, 3]$

State = e , symbol at position 3 = b .

Look up $\delta(e, b) = (g, \sqcup, -1)$.

$\Rightarrow$ State becomes g , write $\sqcup$ at position 3, head moves to 2.

6. **Config 5:** $[g, \sqcup a \sqcup \dots, 2]$

State = g , symbol at position 2 = a .

Look up $\delta(g, a) = (g, a, -1)$.

$\Rightarrow$ State stays g , write a (unchanged), head moves to 1.

7. **Config 6:** $[g, \sqcup a \sqcup \dots, 1]$

State = g , symbol at position 1 = $\sqcup$.

Look up $\delta(g, \sqcup) = (s, \sqcup, +1)$.

$\Rightarrow$ State becomes s , head moves to 2.

8. **Config 7:** $[s, \sqcup a \sqcup \dots, 2]$

State = s , symbol at position 2 = a .

Look up $\delta(s, a) = (h, \sqcup, +1)$.

$\Rightarrow$ State becomes h , write $\sqcup$ at position 2, head moves to 3.

9. **Config 8:** $[h, \sqcup \sqcup \sqcup \dots, 3]$

State = h , symbol at position 3 = $\sqcup$.

Look up $\delta(h, \sqcup) = (e, \sqcup, -1)$.

$\Rightarrow$ State becomes e , head moves to 2.

10. **Config 9:** $[e, \sqcup \sqcup \sqcup \dots, 2]$

State = e , symbol at position 2 = $\sqcup$.

Look up $\delta(e, \sqcup) = (r, \sqcup, +1)$.

$\Rightarrow$ State becomes r . **STOP** — r is the reject state.

Final answer in compact notation: $[s, aab \sqcup \dots, 1] \vdash_M [h, \sqcup ab \sqcup \dots, 2] \vdash_M [h, \sqcup ab \sqcup \dots, 3] \vdash_M [h, \sqcup ab \sqcup \dots, 4] \vdash_M [g, \sqcup a \sqcup \dots, 2] \vdash_M [g, \sqcup a \sqcup \dots, 1] \vdash_M [s, \sqcup a \sqcup \dots, 2] \vdash_M [h, \sqcup \sqcup \dots, 3] \vdash_M [e, \sqcup \sqcup \dots, 2] \vdash_M [r, \sqcup \sqcup \dots, 3]$

Result: aab is **rejected** by M .

This is pure mechanical bookkeeping. No thinking, just: look up (state, symbol) in table $\rightarrow$ write new config. If you can do one step, you can do all of them.

3.3 Common Mistakes in Tracing

Things That Go Wrong When Tracing

1. **Forgetting to update the tape.** When δ says write b , the tape *changes*. In the next step, you read the *new* symbol, not the old one.
2. **Wrong head position.** $+1$ means the head moves *right* (position increases). -1 means it moves *left* (position decreases). Don't mix them up.
3. **Forgetting the left-boundary special case.** If head is at position 1 and direction is -1 , the head *stays* at position 1. This is the only case where the head doesn't move.
4. **Reading the wrong column.** Double-check which symbol is at the current position. After the tape has been modified, the symbol might be different from the original input.

Use your finger (or pen) to physically track the head position on your paper. Draw the tape if it helps. The most common mistake is losing track of where the head is.

4 Type 3: DTM/NTM Construction

What This Type Looks Like

Pattern: Given a language description, build a Turing machine that accepts it.

Where it appears: T1-Ex4, T1-Ex5, T2-Ex4, T3-Ex3

Analogy: This is like being an architect. Someone says “I want a house with 3 bedrooms and a garage.” You don’t just start placing bricks — you first sketch a floor plan (algorithm in words), then draw blueprints (states and transitions), then build (write the δ table). The exercises expect all three phases.

4.1 The Method (Step by Step)

Template: Constructing a DTM

1. Phase 1: Understand the language

- (a) Write out the language formally: $L = \{w \in \Sigma^* \mid \dots\}$
- (b) List 3–4 examples of words IN the language
- (c) List 3–4 examples of words NOT in the language
- (d) **Corner cases!** What about: the empty word ε ? Single-character words? What if the input has unexpected symbols?

2. Phase 2: Algorithm in plain English

- (a) Describe what a human would do to check if a word is in the language
- (b) Use bullet points: “First, look at the first letter. Then, scan right until...”
- (c) Identify: what information do you need to *remember* while scanning? Each piece of remembered info = a state (or set of states)

3. Phase 3: Translate to states

- (a) Each “mode” or “phase” of the algorithm becomes a state
- (b) Each “thing to remember” doubles the states (one state for “remembering 0”, one for “remembering 1”)
- (c) “Scan right past everything” = a self-loop on all input symbols
- (d) “Stop when you see X” = a transition on X that goes to a new state

4. Phase 4: Write the formal definition

- (a) List $Q, \Sigma, \Gamma, s, t, r$
- (b) Write the δ table
- (c) **Fill every cell** — even unreachable ones (put anything, e.g., $(r, \text{symbol}, +1)$)

5. Phase 5: Verify

- (a) Trace the machine on your example words (both in L and not in L)
- (b) Trace on the corner cases
- (c) Check: does it halt on ALL inputs? (If it should be a halting DTM)

4.2 The Key Insight: States = Memory

How to Choose Your States

The fundamental principle: A Turing machine's states are its memory. The tape head can only see ONE cell at a time. If the algorithm needs to “remember” something it saw earlier, that information must be encoded in the current state.

Analogy: You're walking through a dark tunnel with a flashlight (the head). You can only see one tile at a time. If you need to remember the color of the first tile while checking the last tile, you have to carry that information in your head (= in the state).

Translation rules:

Algorithm says...	States become...
“Remember which letter the first symbol was”	q_0, q_1 (one per possible letter)
“Scan right until blank”	One state with self-loops on all non-blank symbols
“Go back to the start”	One state with self-loops moving left
“Check if last letter matches remembered letter”	$check_0, check_1$ (one per remembered letter)
“Mark this cell as visited”	Write a special symbol (like X) on the tape

States = what you remember. Tape modifications = what you write down as notes. Head movement = walking through the input.

4.3 Worked Example: First Letter = Last Letter

Walking Through T1-Ex4 (Full Construction)

Language: Words over $\{0, 1\}$ whose first letter equals their last letter.

Phase 1: Understand the language.

- IN the language: 0, 1, 00, 11, 010, 101, 0110
- NOT in the language: 01, 10, 011, 100, ε
- Corner cases: ε (no first/last letter $\Rightarrow$ reject), single chars like 0 or 1 (first = last $\Rightarrow$ accept)

Phase 2: Algorithm in words.

1. Read the first letter and *remember it*.
2. If there's no first letter (tape is blank), reject.
3. Scan right until you reach the end of the input (first $\sqcup$).
4. Go one step left — you're now on the last letter.
5. Compare the last letter to the remembered first letter.
6. If they match, accept. Otherwise, reject.

Phase 3: Translate to states.

- s : start state. Read first letter.
- q_0 : “I remember the first letter was 0.” Scan right.

- q_1 : “I remember the first letter was 1.” Scan right.
- check_0 : “I’m on the last letter and need it to be 0.”
- check_1 : “I’m on the last letter and need it to be 1.”
- t : accept. r : reject.

Phase 4: Write the δ table.

δ	0	1	$\sqcup$
s	$(q_0, 0, +1)$	$(q_1, 1, +1)$	$(r, \sqcup, +1)$
q_0	$(q_0, 0, +1)$	$(q_0, 1, +1)$	$(\text{check}_0, \sqcup, -1)$
q_1	$(q_1, 0, +1)$	$(q_1, 1, +1)$	$(\text{check}_1, \sqcup, -1)$
check_0	$(t, 0, +1)$	$(r, 1, +1)$	$(r, \sqcup, +1)$
check_1	$(r, 0, +1)$	$(t, 1, +1)$	$(r, \sqcup, +1)$

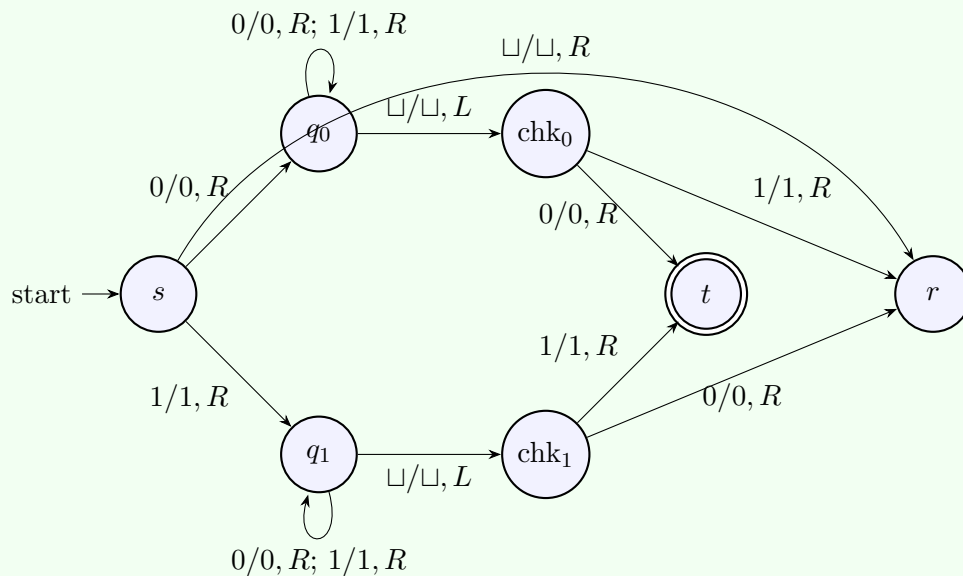
Reading the table row by row:

- s : If first symbol is 0, remember it (go to q_0). If 1, remember it (go to q_1). If blank (empty input), reject.
- q_0/q_1 : Scan right past all input symbols (self-loop). When we hit $\sqcup$, go left into the check state.
- check_0 : We’re on the last letter, and we need it to be 0. If it is 0, accept. If it’s 1, reject. If it’s $\sqcup$ (this means the input was just one character and we’re now left of it — actually this handles the edge case where check lands on blank, meaning reject).

The pattern “remember something, scan to the end, compare” covers a huge number of construction exercises. The structure is always: read $\rightarrow$ remember (via state) $\rightarrow$ scan (self-loop) $\rightarrow$ check.

4.4 State Diagram Visualization

What the Machine Looks Like as a Diagram



The self-loops on q_0 and q_1 are the “scanning” phase. The edges to chk states are “I reached

the end.” The edges from chk to t or r are “compare and decide.”

4.5 The Answer Format the Grader Expects

What to Actually Write on Your Exam Paper

Based on the solutions, a DTM construction answer has **exactly these parts** in this order:

1. **Formal language definition** (if asked): $L = \{a_1 a_2 \cdots a_n \mid a_j \in \{0, 1\}, n \geq 1, a_1 = a_n\}$
2. **Intuitive description** (2–4 bullet points): “The DTM works as follows:”
 - Store the first letter using states q_0 and q_1
 - Scan right until blank
 - Go left and compare
3. **Formal definition:** “We define $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with:”
 - $Q = \{\dots\}$
 - $\Sigma = \{\dots\}$
 - $\Gamma = \{\dots\}$
 - δ given by the following table: [the table]
4. **Runs on example inputs** (if asked): Use the $\vdash_M$ notation.

Always give the intuition BEFORE the formal definition. The grader wants to see that you understand what the machine does, not just that you can fill in a table.

4.6 NTM Construction: When and How

DTM vs NTM: When to Use Which

Use a DTM when: The algorithm is deterministic — at each step, there’s only one thing to do. (T1-Ex4, T1-Ex5)

Use an NTM when: The problem requires *guessing* something — like where to split a string, or which nondeterministic branch to take. (T2-Ex4, T3-Ex3)

The NTM trick: Instead of trying all possibilities one by one, an NTM tries them all *parallel*. If any branch accepts, the whole machine accepts.

Translation rule:

- “Try all possible split points” $\Rightarrow$ NTM: at each position, nondeterministically choose “split here” or “continue”
- “Guess two numbers” $\Rightarrow$ NTM: nondeterministically write two numbers, then verify
- “Check if infix XYZ exists” $\Rightarrow$ NTM: nondeterministically guess where XYZ starts, then verify

NTM δ table format: Instead of $\delta(q, a) = (q', b, d)$, you write $\delta(q, a) = \{(q'_1, b_1, d_1), (q'_2, b_2, d_2)\}$ — a *set* of possible transitions.

NTM = “guess and verify.” The nondeterminism handles the guessing; after the guess, the verification is usually deterministic.

5 Type 4: Simulation / Robustness Proofs

What This Type Looks Like

Pattern: “Show that variant X of DTMs can be simulated by standard DTMs.” Or equivalently: “Show that adding feature X doesn’t make DTMs more powerful.”

Where it appears: T2-Ex1 (stay direction), T2-Ex2 (doubly-infinite tape)

Analogy: Someone claims their fancy new car (DTM variant) can reach places that a normal car can’t. You need to show that a normal car can reach all the same places — maybe by taking a slightly different route, but it gets there.

5.1 The Method

Template: Simulation Proof

1. **Identify the difference:** What can the variant do that a standard DTM can’t? (e.g., “stay in place” or “move left past cell 1”)
2. **Show how to fake it:** How can a standard DTM achieve the same effect using only its standard operations?
 - “Stay” = go right then go left (2 steps instead of 0, but same result)
 - “Doubly-infinite tape” = shift content when hitting the left boundary
3. **Construct M' formally:**
 - $M' = (Q', \Sigma, \Gamma', s, t, r, \delta')$ — same Σ, s, t, r
 - $Q' = Q \cup \{\text{auxiliary states}\}$ — add new states for the workaround
 - δ' = copy of δ for normal transitions, plus new transitions for the workaround
4. **State the correctness claim:**

“ M' outcome-simulates M : if M halts/accepts on input w , then M' halts/accepts on input w .”
5. **Argue why it’s correct** (usually obvious from the construction, but say it explicitly).

5.2 Worked Example: The “Stay” Direction

How to Simulate “Stay” (T2-Ex1)

The difference: The variant allows $\delta(q, a) = (q', b, 0)$ — don’t move the head.

How to fake it with a standard DTM:

Go right one step, then go left one step. You end up at the same cell.

New states needed: For each state q' that should be the target of a “stay” transition, add an auxiliary state q'_L that means “I need to go left to get back to where I was.”

New transitions:

- If the original has $\delta(q, a) = (q', b, 0)$ (stay), replace with:
 - $\delta'(q, a) = (q'_L, b, +1)$ — write b , go right, remember we need to go back
 - $\delta'(q'_L, c) = (q', c, -1)$ for all $c \in \Gamma$ — whatever we see, leave it unchanged, go left, arrive at q'
- If the original has $\delta(q, a) = (q', b, +1)$ or $(q', b, -1)$ (normal), copy unchanged: $\delta'(q, a) = \delta(q, a)$

Why it's correct: After the two moves (right then left), the head is back at the original position, in state q' , with symbol b written — exactly the same as the “stay” transition. The cell we visited in between is left unchanged.

The simulation pattern is: identify the “exotic” operation $\rightarrow$ replace it with a sequence of standard operations that achieves the same effect $\rightarrow$ add auxiliary states to manage the extra steps.

6 Type 5: Spot the Error in Proofs

What This Type Looks Like

Pattern: Given a “proof” that’s wrong, find the error and explain why.

Where it appears: T3-Ex2

Analogy: Think of it like proofreading a recipe. The recipe claims to make chocolate cake, but you know from experience that the recipe produces something else. Your job: find the step where they added salt instead of sugar.

6.1 The Method

Template: Spotting Errors in Proofs About CE/Computable

The most common errors in proofs about CE languages:

1. **The “Simulate M on w ” trap:** If M doesn’t halt on w , then the simulation *never finishes*. So any code that says “Simulate M on w . If M rejects, then do X ” — the “do X ” part is *never reached* if M loops. The proof implicitly assumes M halts, but for CE languages, M might not halt.
2. **The “detect non-halting” trap:** A proof says “If M does not halt on w , then M' accepts.” This is nonsense — there is no way for a DTM to detect that another DTM doesn’t halt. That would solve the halting problem.
3. **The “swap accept/reject” trap:** Swapping t and r for a CE language’s DTM does NOT give you the complement. If M loops on $w \notin L$, then swapping t and r still causes M' to loop on $w \notin L$ — so M' doesn’t accept w either. You end up with $L(M') \neq \bar{L}$.

Detection strategy: For each step in the “proof,” ask yourself:

- Does this step assume M halts? (If M is only CE, it might not halt.)
- Does this step try to detect non-halting? (Impossible.)
- Try a **concrete counterexample**: pick a DTM M that loops on some input and trace through the “proof.”

The Classic Counterexample for “Swap t and r ”

The flawed proof says: “Given CE language L with DTM M , construct M' that accepts when M rejects and vice versa. Then $L(M') = \bar{L}$.”

Counterexample: Let M be a DTM over $\Sigma = \{a\}$ where $\delta(s, a) = (s, a, +1)$ and $\delta(s, \sqcup) = (s, a, +1)$.

This machine *never halts* on any input. So $L(M) = \emptyset$.

Now build M' by swapping t and r . But M never reaches t or r — it loops forever. So M' also loops forever on every input. Therefore $L(M') = \emptyset$.

But $\bar{L} = \bar{\emptyset} = \Sigma^*$, and $L(M') = \emptyset \neq \Sigma^*$.

The swap trick fails because looping inputs remain looping. M loops $\Rightarrow M'$ loops (swapping t and r doesn’t help if neither is ever reached).

7 Type 6: Closure Properties

What This Type Looks Like

Pattern: “Prove that the class of computable/CE languages is closed under [union/intersection/concatenation/complement/...].”

Where it appears: T2-Ex5, T3-Ex5

Analogy: If you have two cookie cutters (DTMs for L_1 and L_2) and someone asks “can you make a cookie that satisfies BOTH shapes?” — you need to show how to combine the two cutters into one that does the job.

7.1 The Method

Template: Closure Proof

1. **Start with:** “Let L_1 and L_2 be [computable/CE] languages. By definition, there exist [halting] DTMs M_1 and M_2 such that $L(M_1) = L_1$ and $L(M_2) = L_2$.”
2. **Construct** M that accepts the combined language ($L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 \cdot L_2$, etc.):
 - **Union:** “On input w : Simulate M_1 on w . If it accepts, accept. Otherwise, simulate M_2 on w . If it accepts, accept. Otherwise, reject.” (For CE: use parallel simulation — run both step-by-step.)
 - **Intersection:** “On input w : Simulate M_1 on w . If it rejects, reject. Otherwise, simulate M_2 on w . Accept iff M_2 accepts.”
 - **Complement:** “On input w : Simulate M_1 on w . If it accepts, reject. If it rejects, accept.” (**Only works for computable languages!** M_1 must halt.)
 - **Concatenation:** Use NTM: “Nondeterministically split $w = w_1w_2$. Simulate M_1 on w_1 and M_2 on w_2 . Accept if both accept.”
3. **Argue correctness:** “ M accepts w iff [both/either] M_1 and M_2 accept their inputs, which happens iff $w \in L_1[\text{op}] L_2$.”
4. **Argue halting** (if proving closure for computable languages): “ M halts because M_1 halts (it’s a halting DTM) and M_2 halts.”

The Critical Difference: Computable vs CE Closure

Why complement closure fails for CE:

For **computable** languages, M is halting, so “simulate M and swap the answer” works.

For **CE** languages, M might loop. If M loops, you can’t “swap” — you never get an answer to swap. That’s why CE is NOT closed under complement.

Why union needs special handling for CE:

For **computable**: Run M_1 , then M_2 . Since both halt, this terminates.

For **CE**: If you run M_1 first and it loops, you never get to run M_2 — even if M_2 would accept!

Fix: parallel simulation (dovetailing). Run one step of M_1 , one step of M_2 , one step of M_1 , one step of M_2 , ... Accept as soon as either one accepts.

For computable: sequential simulation is fine (both halt). For CE: you often need parallel simulation (dovetailing) to avoid getting stuck in a loop.

8 Type 7: Computability Classification

What This Type Looks Like

Pattern: Given a language L , determine: is it computable, CE-but-not-computable, or not even CE? If computable, describe a halting DTM.

Where it appears: T4-Ex1, T4-Ex2

Analogy: Think of it like a border patrol officer. Someone hands you a passport (a language). You need to decide: “Is this person allowed in (computable)?” or “They might be allowed but I can’t tell for sure (CE)?” or “Definitely not allowed (not CE)?” Your tool is: what can a DTM *actually check* in finite time?

8.1 The Decision Flowchart

Template: Is L Computable?

Ask yourself these questions in order:

1. Is the property “finite” or “bounded”?

- “ M has more than 5 states” $\rightarrow$ just count the states in the encoding. **Computable.**
- “ M accepts w in less than 1000 steps” $\rightarrow$ simulate for 1000 steps and check. **Computable.**
- Key insight: if you can put a *fixed upper bound* on the computation you need to do, it’s computable.

2. Is it a syntactic property of the encoding?

- Properties of the *encoding* $[M]$ (number of states, alphabet size, specific transitions) can be checked by just parsing the string. **Computable.**

3. Does it require running M for an unbounded number of steps?

- “ M accepts w ” = “ M accepts w in a **finite** number of steps” = AP. **Not computable.**
- “ M halts on w ” = HP. **Not computable.**
- “ $L(M) \neq \emptyset$ ” = NEP. **Not computable.**

4. Is it a disguised version of a known problem?

- “ M accepts w in a finite number of steps” = “ M accepts w ” = AP. The word “finite” is a red herring — accepting *always* takes finitely many steps.
- “ M halts in accepting or rejecting state” = “ M halts” = HP. A DTM can *only* halt in t or r , so this adds nothing.

Worked Example: Classifying T4-Ex1 Languages

$L_1 = \{ \langle [M] \rangle \mid M \text{ has more than 5 states} \}$

Property of the encoding (syntactic). Just parse the encoding and count states.

$\Rightarrow$ **Computable.** DTM: check that the first 6 symbols of the encoding are all 1’s.

$L_2 = \{ \langle [M], w \rangle \mid M \text{ accepts } w \text{ in less than 1000 steps} \}$

Bounded computation. Simulate M on w for exactly 1000 steps.

$\Rightarrow$ **Computable.** DTM: simulate for 1000 steps, accept if M accepted during simulation, reject otherwise.

$L_3 = \{\langle [M], w \rangle \mid M \text{ accepts } w \text{ in a finite number of steps}\}$

Trap: “finite number of steps” sounds bounded, but it’s not. “ M accepts w in a finite number of steps” is exactly the same as “ M accepts w ” (because acceptance always takes finitely many steps). This is exactly AP.

$\Rightarrow$ **Not computable.**

$L_4 = \{\langle [M], w \rangle \mid M \text{ halts in } t \text{ or } r\}$

Trap: A DTM can *only* halt in t or r (those are the only halting states). So “halts in t or r ” is exactly “halts.” This is HP.

$\Rightarrow$ **Not computable.**

The key skill here is recognizing disguised versions of AP and HP. If it boils down to “does M accept w ?” or “does M halt on w ?” without a bound, it’s not computable.

8.2 Writing the DTM for Computable Cases

Template: Describing a Halting DTM for a Computable Language

The expected answer format:

1. State that the language is computable.
2. Describe the DTM: “Consider the following halting DTM M :”
 - “On input w :”
 - Step 1: Check if input has the right format (if it involves encodings)
 - Step 2: Do the bounded check
 - Step 3: Accept or reject based on the result
3. Argue that the DTM halts: “This DTM halts because [the simulation is bounded / we only check a fixed number of symbols / etc.]”

You do NOT need to give a full δ table for these. A clear algorithmic description is sufficient, as long as it’s obvious that a DTM could implement it.

9 Type 8: Reduction Proofs

What This Type Looks Like

Pattern: “Show that language L is not computable by reducing from a known non-computable language.”

Where it appears: T4-Ex5

Analogy: You want to prove that Problem B is hard. You know Problem A is hard. So you show: “If someone could solve B, they could also solve A — but we know A can’t be solved. Therefore B can’t be solved either.” It’s a proof by contradiction using a translation.

9.1 What a Reduction Actually Is

Many-One Reduction ($\leq_m$)

Plain English:

A reduction from A to B is a computable function f that translates every input for A into an input for B , in a way that preserves the answer. If x is a yes-instance of A , then $f(x)$ is a yes-instance of B . If x is a no-instance of A , then $f(x)$ is a no-instance of B .

Formal Definition: $A \leq_m B$ if there exists a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that $x \in A \iff f(x) \in B$.

What it means:

- If $A \leq_m B$ and A is not computable, then B is not computable either.
- Intuition: B is *at least as hard* as A .
- Direction matters: you reduce FROM the known hard problem TO the new problem.

9.2 The Method

Template: Reduction Proof (“ L is not computable”)

1. **Choose your known non-computable language.** Usually AP (acceptance problem) or HP (halting problem).
2. **Define the reduction function f .** Given an input to the known problem, construct an input to L .
The most common pattern: given $\langle [M], w \rangle$, construct a new DTM $\mathcal{E}_{M,w}$ whose behavior depends on whether M accepts w .
3. **The “ $\mathcal{E}_{M,w}$ construction” (the master template):**
 - Define a new DTM $\mathcal{E}_{M,w}$ that works as follows:
 - “On input x (which $\mathcal{E}_{M,w}$ ignores):”
 - (a) Erase x from the tape.
 - (b) Write w on the tape.
 - (c) Simulate M on w .
 - (d) If M accepts, then [do whatever makes $\mathcal{E}_{M,w}$ satisfy the target property].
 - Key insight: $\mathcal{E}_{M,w}$ ignores its own input! Its behavior depends entirely on whether M accepts w .
4. **Show the equivalence:**
 - If $\langle [M], w \rangle \in \text{AP}$, then M accepts w , so $\mathcal{E}_{M,w}$ does [something], so $[\mathcal{E}_{M,w}] \in L$.

- If $\langle [M], w \rangle \notin \text{AP}$, then M doesn't accept w , so $\mathcal{E}_{M,w}$ doesn't do [something], so $[\mathcal{E}_{M,w}] \notin L$.

5. **Define** f : $f(x) = \begin{cases} [\mathcal{E}_{M,w}] & \text{if } x = \langle [M], w \rangle \\ \varepsilon & \text{otherwise} \end{cases}$

6. **Conclude**: “We have shown $\text{AP} \leq_m L$. Since AP is not computable, L is not computable either.”

9.3 Worked Example: NEP is Not Computable

Walking Through T4-Ex5

Goal: Show that $\text{NEP} = \{ [M] \mid L(M) \neq \emptyset \}$ is not computable.

Step 1: Reduce from $\text{AP} = \{ \langle [M], w \rangle \mid M \text{ accepts } w \}$.

Step 2: Given $\langle [M], w \rangle$, define $\mathcal{E}_{M,w}$:

“On input x :

1. Erase x from the tape.
2. Write w on the tape.
3. Simulate M on w .
4. If M accepts w , then $\mathcal{E}_{M,w}$ accepts (if M rejects, $\mathcal{E}_{M,w}$ rejects too).”

Step 3: The key observation.

- $\mathcal{E}_{M,w}$ ignores its input x . It always simulates M on w .
- If M accepts w : then $\mathcal{E}_{M,w}$ accepts *every* input $\Rightarrow L(\mathcal{E}_{M,w}) = \Sigma^* \neq \emptyset \Rightarrow [\mathcal{E}_{M,w}] \in \text{NEP}$.
- If M does not accept w : then $\mathcal{E}_{M,w}$ accepts *no* input $\Rightarrow L(\mathcal{E}_{M,w}) = \emptyset \Rightarrow [\mathcal{E}_{M,w}] \notin \text{NEP}$.

Step 4: Therefore: $\langle [M], w \rangle \in \text{AP} \iff [\mathcal{E}_{M,w}] \in \text{NEP}$

Define $f(x) = [\mathcal{E}_{M,w}]$ if $x = \langle [M], w \rangle$, and $f(x) = \varepsilon$ otherwise.

The function f is computable (it just manipulates the encoding). So $\text{AP} \leq_m \text{NEP}$. Since AP is not computable, NEP is not computable either.

The $\mathcal{E}_{M,w}$ trick works for many reductions: build a machine that ignores its own input and instead simulates M on w . Then the target property of $\mathcal{E}_{M,w}$ depends entirely on whether M accepts w .

10 Type 9: CE vs Computable Arguments

What This Type Looks Like

Pattern: Show that a language is CE (but possibly not computable), or use the theorem “ L is computable iff both L and $\bar{L}$ are CE.”

Where it appears: T3-Ex4 (HP is CE), T4-Ex3 (the iff theorem), T4-Ex4 ($\overline{AP}$ is not CE)

10.1 The Three Key Tools

The Three Tools for CE/Computable Arguments

Tool 1: “Simulate and see what happens” (showing something is CE).

To show L is CE, construct a DTM that accepts $w \in L$ (but may loop on $w \notin L$).

Template: “On input w : Simulate [something]. If it accepts/halts, accept.”

The machine doesn’t need to handle the non-membership case — looping is fine for CE.

Tool 2: The Complement Theorem.

L is computable $\iff$ both L and $\bar{L}$ are CE.

To prove $\Rightarrow$: If L is computable, then both L and $\bar{L}$ have halting DTMs. Halting DTMs are special cases of DTMs, so both are CE.

To prove $\Leftarrow$: Run the DTM for L and the DTM for $\bar{L}$ in parallel (dovetailing). One of them *must* accept (since every w is in either L or $\bar{L}$). If the L -machine accepts first, accept. If the $\bar{L}$ -machine accepts first, reject.

Tool 3: Proof by contradiction (showing something is NOT CE).

“Assume $\bar{L}$ is CE. Then both L and $\bar{L}$ are CE. By Tool 2, L is computable. But we know L is not computable. Contradiction. So $\bar{L}$ is not CE.”

10.2 Worked Example: HP is CE

Walking Through T3-Ex4

Goal: Show $HP = \{\langle \lceil M \rceil, w \rangle \mid M \text{ halts on } w\}$ is CE.

Construct the DTM $\mathcal{H}$:

“On input x :

1. If x is not of the form $\langle \lceil M \rceil, w \rangle$, reject.
2. Otherwise, simulate M on w .
3. If M halts (either accepts or rejects), then accept.”

Why this works:

- If $\langle \lceil M \rceil, w \rangle \in HP$ (i.e., M halts on w): the simulation eventually halts, so $\mathcal{H}$ accepts. Good.
- If $\langle \lceil M \rceil, w \rangle \notin HP$ (i.e., M doesn’t halt on w): the simulation runs forever, so $\mathcal{H}$ doesn’t halt. That’s fine — for CE, we only need to accept members, not reject non-members.

Showing something is CE is usually easy: just simulate and accept if the simulation terminates successfully. The hard part of CE problems is usually showing something is NOT CE (which requires the complement theorem or a reduction).

10.3 Worked Example: $\overline{AP}$ is Not CE

Walking Through T4-Ex4

Goal: Show $\overline{AP}$ is not CE.

Proof (3 lines):

1. Assume towards contradiction that $\overline{AP}$ is CE.
2. We know AP is CE (proven in lecture/similar to HP being CE).
3. Then both AP and $\overline{AP}$ are CE, so by the Complement Theorem, AP is computable.
4. But we proved in lecture that AP is NOT computable. Contradiction.
5. Therefore $\overline{AP}$ is not CE.

This proof pattern is extremely common: if you know L is CE but not computable, then $\overline{L}$ cannot be CE (otherwise L would be computable by the complement theorem).

11 Putting It All Together: Your Study Plan

The Concrete Steps to Take

Phase 1: Definitions (30 minutes).

Write out the 5 definitions from Section 1.3 on a blank piece of paper, from memory, 3 times. If you get stuck, look once, then try again. Don't move to Phase 2 until you can write all 5 without looking.

Phase 2: Mechanical Tracing (45 minutes).

Take the DTM from T1-Ex3 and trace it on inputs: ab , ba , bb , a , ε . Don't look at solutions. Just follow the δ table mechanically. This builds the "muscle memory" for the notation.

Phase 3: One Construction (60 minutes).

Pick a language you haven't seen before (make one up: "words that start and end with 0" or "words with even length"). Follow the 5-phase template from Section 4. Write the full DTM. Trace it on 3 inputs to verify.

Phase 4: One Proof (45 minutes).

Try the closure proof for union: "computable languages are closed under union." Follow the template from Section 7. Write it out in full.

Phase 5: One Reduction (60 minutes).

Try to show that $TOTAL = \{[M] \mid M \text{ halts on every input}\}$ is not computable. Use the reduction template from Section 9. (Hint: reduce from HP.)

Total: about 4 hours. After this, every exercise in Tutorials 1–4 should feel like a variation of something you've already done.

11.1 Quick Reference: Which Template for Which Exercise

Exercise asks you to...	Use Type	Section
"Is this statement correct?"	Definition Checking	§2
"Is this claim true/false?"	Definition Checking + Counterexample	§2.4
"Give the run of M on w "	Mechanical Tracing	§3
"Construct a DTM for L "	DTM Construction	§4
"Construct an NTM for L "	NTM Construction	§4.6
"Show variant can be simulated"	Simulation / Robustness	§5
"Find the error in this proof"	Spot the Error	§6
"Show class closed under X "	Closure Properties	§7
"Is L computable?"	Computability Classification	§8
"Show L is not computable"	Reduction	§9
"Show L is (not) CE"	CE vs Computable	§10

11.2 The Answer Format Cheat Sheet

How Long Should Your Answers Be?

Based on the official solutions:

- **Definition checking:** 2–3 sentences. State the answer, cite the definition, done.
- **Tracing:** Just write the chain of configurations. No explanation needed.

- **DTM construction:** Intuitive description (3–5 bullet points) + formal definition ($Q, \Sigma, \Gamma, \delta$ table) + runs on examples.
- **Simulation:** Intuitive idea (1 paragraph) + formal construction of M' + correctness argument (1 paragraph).
- **Closure proof:** 1 paragraph setup + DTM description (numbered steps) + 1 paragraph arguing correctness and halting.
- **Reduction:** Construction of $\mathcal{E}_{M,w}$ + equivalence argument ($x \in A \iff f(x) \in B$) + conclusion.
- **CE argument:** Usually 3–5 lines total.

Most answers are shorter than you think. The solutions are precise and concise — they don't ramble. Say exactly what needs to be said, cite the right definition or construction, and stop.